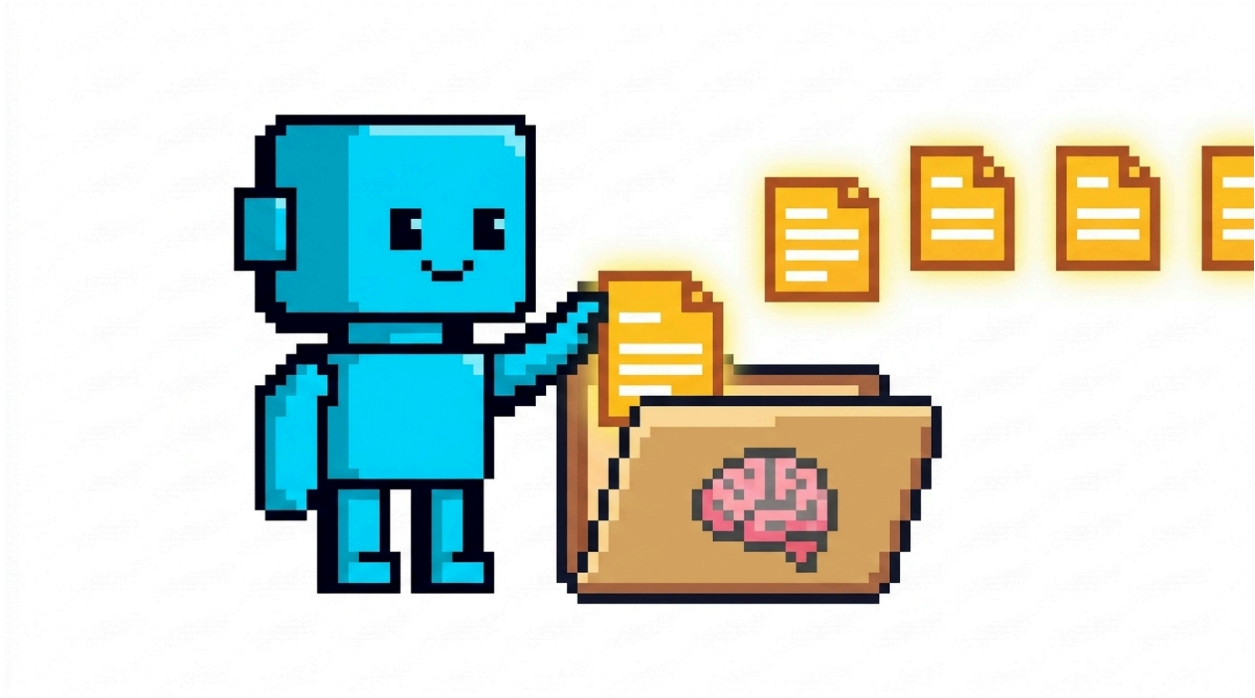




The Second Brain Blueprint



How to build the AI operating system that runs a business, starting from an empty folder

QUICK TIP: WITH FABLE 5 YOU CAN COPY THIS ENTIRE DOCUMENT AND PASTE INTO CLAUDE CODE.

ADD THIS PROMPT

“This is a SOP on how to build a second brain. Walk me through the process”

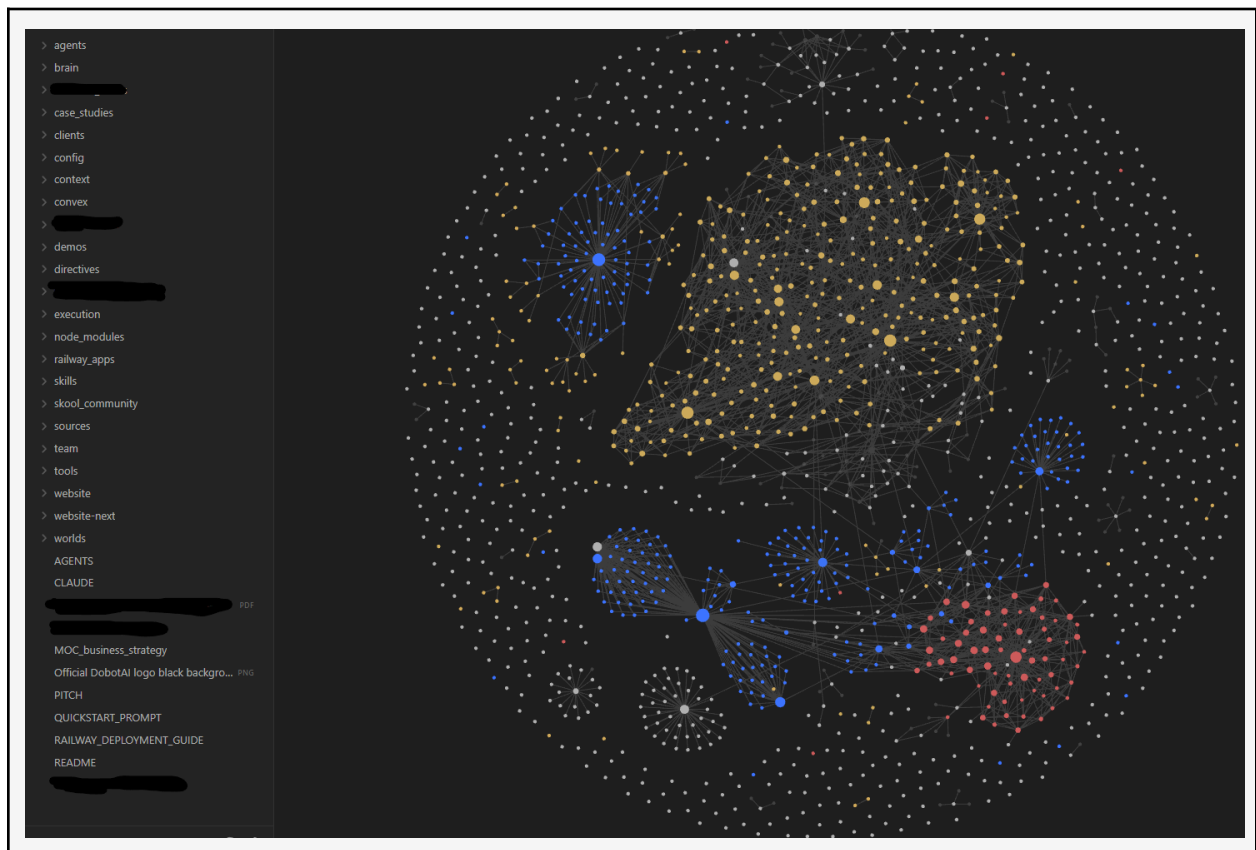
Author: Doby Lanete, DobotAI

Date: 2026-07-05

What this is

This is the exact blueprint for the system that runs my agency, DobotAI. As of today the real thing contains:

- 168 standard operating procedures the AI can execute
- 189 automation scripts it calls to do the actual work
- 297 skill files holding deep domain expertise
- 72 linked notes covering every decision, client, and lesson in the business
- 47 client folders with profiles, rules, preferences, and history



The entire company fits in one folder. An AI agent reads it, understands the business better than a new hire would after six months, and does real work inside it every day: content, research, client deliverables, sales prep, follow-ups.

Here is the part that matters. AI models get replaced constantly. The model I used to build most of this system is being retired. And it changes nothing, because the asset is the folder. Whatever model comes next reads the same files and picks up where the last one left off, the same day.

This guide shows you how to build that folder from scratch. Every phase includes what to create, why it exists, and a template you can copy.

Who this is for: business owners who want AI to actually operate parts of their business, not just answer questions in a chat window.

What you need: a computer, Claude Code (or any capable AI coding agent), Obsidian (free, it is how you will read the brain you build), and about a week of honest effort.

Part 1: The architecture (read this before building anything)

Most people use AI wrong. They open a chat, type a request, get an answer, and lose everything when the conversation ends. Every session starts from zero. The AI never learns your business, your voice, or your standards.

A second brain fixes this by separating the system into three layers. DOE: Directive, Orchestration, Execution.

DIRECTIVES	(markdown files)	-> What to do: step-by-step SOPs in plain English
ORCHESTRATION	(the AI agent)	-> The decision maker: reads context, picks the SOP, checks quality
EXECUTION	(scripts)	-> How it gets done: deterministic code for API calls and file work

Why the split matters: AI models are probabilistic. If a model is 90 percent accurate per step, a five-step task done entirely by the model succeeds only 59 percent of the time. So you push everything that can be deterministic (API calls, formatting, file operations, data processing) into scripts that work the same way every time, and you let the AI do only what it is uniquely good at: reading context, making judgment calls, and checking quality.

The second principle: **plain text is forever**. Everything in the system is a markdown file. No proprietary database, no vendor lock-in, no app that can shut down. Text files survive every model migration, every tool change, every platform shift. That is why the infrastructure outlives the model.

The third principle: **the system improves itself**. Every time something breaks or a better approach appears, the AI updates its own SOPs. I call this self-annealing. A one-time setup becomes a compounding asset.

Part 2: The build, phase by phase

Phase 1: The foundation

Create one folder. This folder is the business brain. Mine looks like this:

```
your-business-brain/
├── CLAUDE.md          - The operating manual the AI reads first, every
session              -
├── context/           - Who you are: identity, voice, values, services
├── directives/        - SOPs: what to do, step by step
├── execution/         - Scripts: the deterministic work
├── skills/            - Deep domain expertise files
├── clients/           - One folder per client
├── brain/             - Linked notes: decisions, history, lessons
├── sources/           - Raw exports the brain is built from
└── .tmp/              - Scratch space for drafts (never committed)
```

Then two setup steps:

1. **Initialize git.** Run `git init` in the folder. Version control means every change the AI makes is tracked and reversible. This is non-negotiable: you are about to let an AI edit your business's memory, and git is the undo button.

2. **Create a .env file for API keys** and add it to `.gitignore` immediately. Keys never get committed. Add `.tmp/` to `.gitignore` too.

Action step: create the folder, the empty subfolders, run `git init`, create `.gitignore` with `.env` and `.tmp/` in it.

Phase 2: Teach it who you are (the context layer)

The `context/` folder is loaded before any work happens. It is the difference between generic AI output and output that sounds like you and serves your strategy.

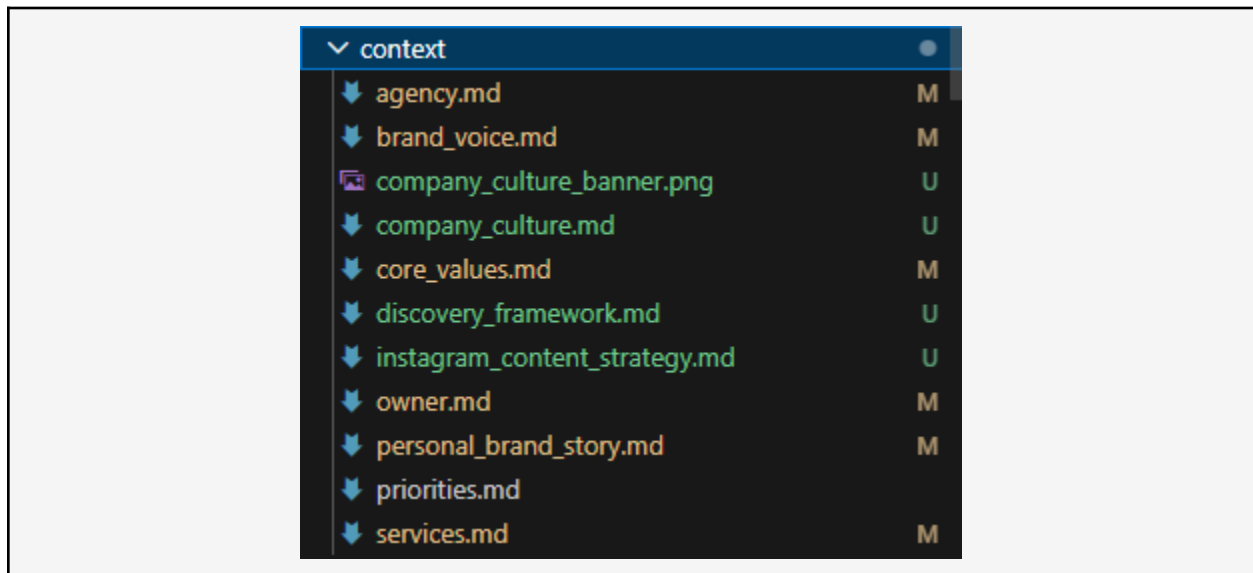
Create these four files first:

context/agency.md (or `company.md`): who you are, what you sell, who you serve, how you are positioned, real results you can prove, and current-state facts like team and stage. Write it like an onboarding doc for a sharp new hire.

context/brand_voice.md: how you sound. Tone, formality level, words you use, words you ban, formatting rules, one good example and one bad example. Be specific enough that two different models would produce recognizably similar output.

context/core_values.md: how you operate. Not poster values. Written as rules with a test for each one, so the AI can check its own work against them.

context/owner.md: your background, expertise, story, and the personal facts that show up in content and sales conversations.



Notice I have a lot more than just the four files but you can start with just the four for now.

The fastest way to write these is not to write them. Record yourself talking for 20 minutes about your business, transcribe it, hand the transcript to the AI, and have it draft the files. Then correct what it got wrong. The corrections are where the real value gets encoded.

Action step: create the four context files. Imperfect drafts today beat perfect drafts never. The system will refine them.

Phase 3: Write the operating manual (CLAUDE.md)

CLAUDE.md sits at the root of the folder and is read automatically at the start of every session. It is the constitution of the system. Mine contains:

1. **The architecture explanation** so the AI knows the DOE pattern and respects the layer boundaries
2. **A directory map** with the purpose of each folder

- 3. **Context loading priority:** which files to read, in what order, before any task
- 4. **The orchestration flow:** parse the request, find the matching SOP, load context, execute, check quality, deliver
- 5. **Standing rules:** things like "never fabricate numbers or client results, use placeholders and ask" and formatting requirements
- 6. **The self-annealing protocol:** after every task, if something broke, fix the script and update the SOP

My context loading priority, which you can copy directly:

- 1. context/agency.md -> Always first (who we are)
- 2. context/core_values.md -> Always (how we operate; check work against it)
- 3. context/brand_voice.md -> For any content creation
- 4. clients/{name}/*.md -> For client-specific work
- 5. skills/relevant files -> Domain expertise for the task
- 6. directives/the SOP -> The workflow itself

Context Loading Priority - On-Disk Verification

verified 2026-07-06

Repo: C:\Users\arne1\OneDrive\Desktop\DobotAI\Agents\Doby2 (branch: main) - Doby2 AI Agency Operating System (DOE architecture)

1 Source of truth: the rule as written in the orchestrator instructions

CLAUDE.md, lines 83-92 (verbatim)

VERIFIED

83

Context Loading Priority

84

85

86

1. context/agency.md

→ Always first (who we are)

87

2. context/core_values.md

→ Always (the law: how we operate; check work against it)

88

3. context/brand_voice.md

→ For any content creation

89

4. clients/{name}/*.md

→ For client-specific work

90

5. skills/SKILL_BIBLE_*

→ Domain expertise for the task

91

6. directives/*.md

→ The workflow SOP itself

92

83

84

85

86

87

88

89

90

91

92

2 The referenced files exist on disk

Get-ChildItem context*.md (live output)

VERIFIED

agency.md

5,413 bytes

modified 2026-07-04 16:08

+ priority 1

core_values.md

4,819 bytes

modified 2026-07-02 18:31

+ priority 2

brand_voice.md

3,766 bytes

modified 2026-07-06 09:26

+ priority 3

company_culture.md

32,620 bytes

modified 2026-07-03 14:11

discovery_framework.md

4,003 bytes

modified 2026-06-25 12:00

instagram_content_strategy.md

4,154 bytes

modified 2026-07-04 16:14

owner.md

3,281 bytes

modified 2026-07-04 16:14

personal_brand_story.md

12,293 bytes

modified 2026-07-04 16:00

priorities.md

3,756 bytes

modified 2026-06-17 18:08

services.md

13,787 bytes

modified 2026-07-04 16:00

48

client folders (priority 4)

clients/{name}\profile.md, rules.md, ...

298

skill bibles (priority 5)

skills\SKILL_BIBLE_*.md

168

workflow SOPs (priority 6)

directives/*.md

Why this is binding:

CLAUDE.md is checked into the repo and injected as project instructions at the start of every Claude Code session. The orchestrator (Claude) is instructed to load context in the order above before executing any directive: agency identity and core values always, brand voice for content work, then client files, skill bibles, and finally the directive itself.

Action step: write CLAUDE.md. Start small: architecture, directory map, loading priority, and three standing rules. It grows with the system.

Phase 4: The work engine (directives and execution)

This is where the system starts doing work instead of just knowing things.

A **directive** is an SOP in markdown: what the workflow is, what it needs, the steps in order, and quality gates the output must pass. One file per workflow. Name them by what they do: weekly_content_plan.md, client_onboarding.md, discovery_call_prep.md.

A directive template that mirrors mine:

[Workflow Name]

What this workflow is

One paragraph: what it produces and when to use it.

Prerequisites

Required API keys, required context files, required skill files.

Inputs

Field	Required	Description

Process

Step 1: [Name]

Step 2: [Name]

Step 3: [Name]

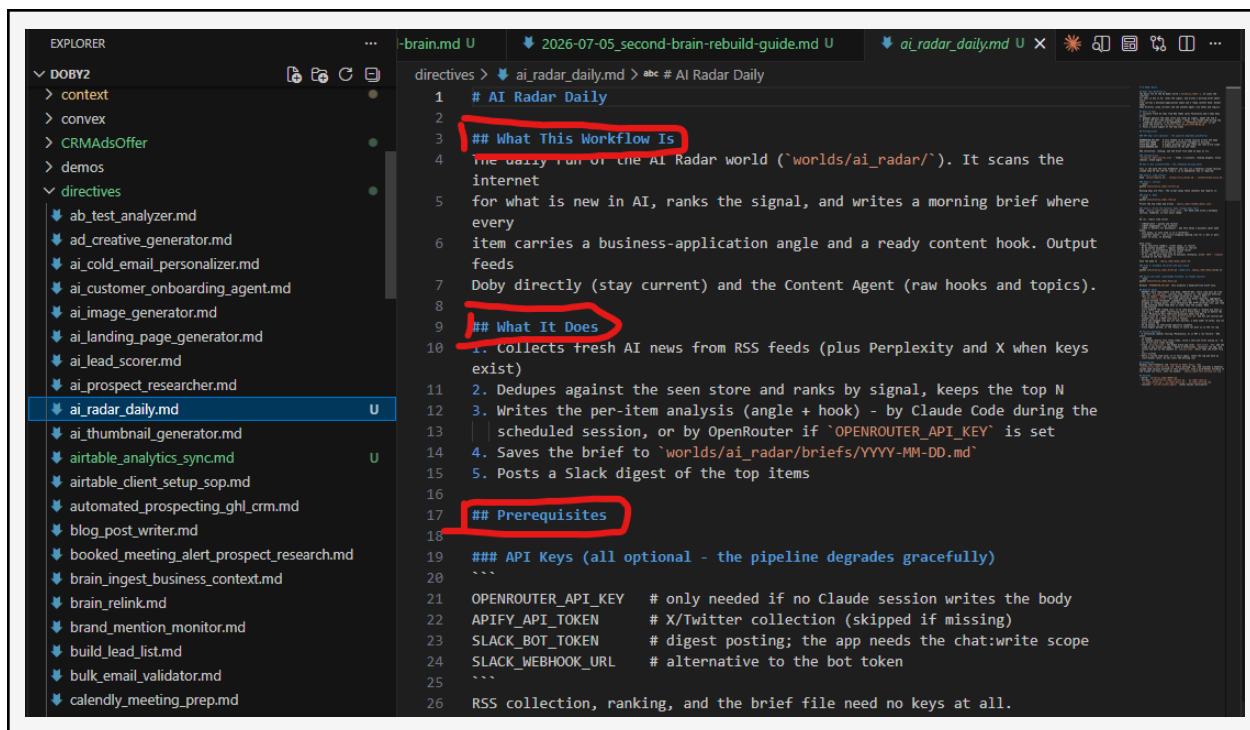
Quality gates

- [] Check 1

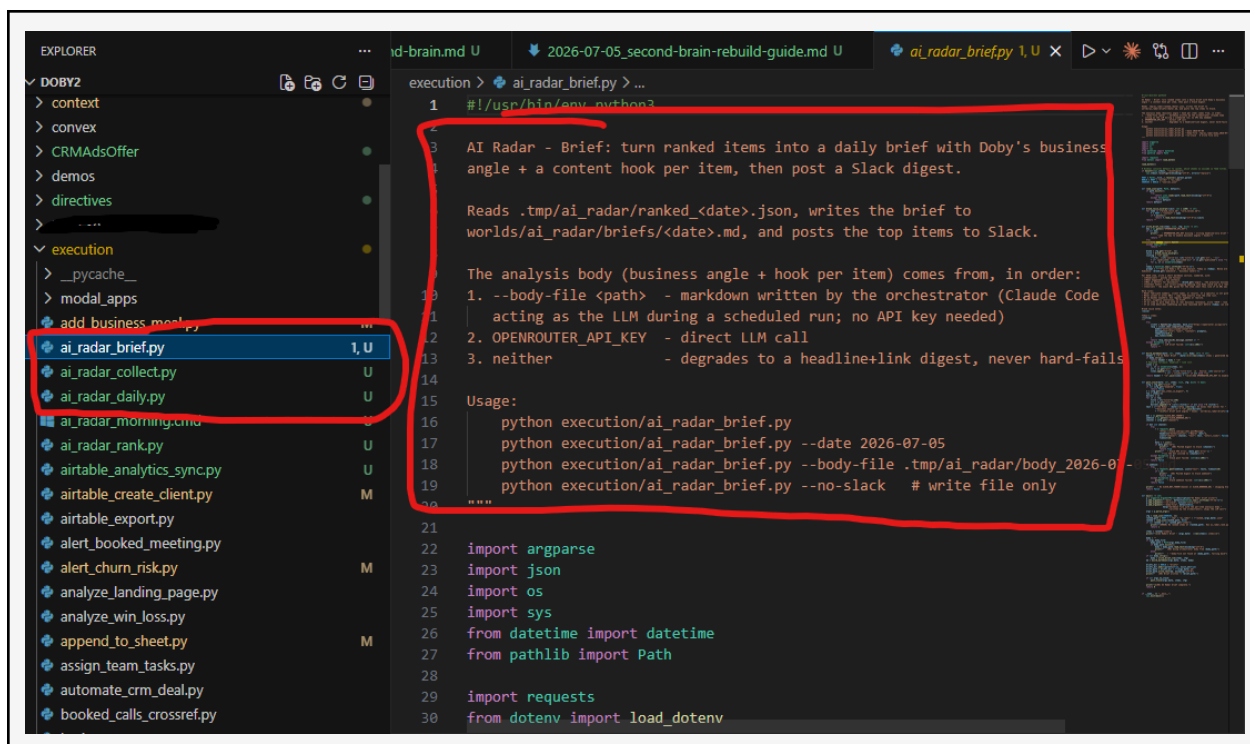
- [] Check 2

Edge cases

- Edge case -> what to do



An **execution script** is code the AI calls when a step should be deterministic: pulling data from an API, formatting a document, sending a notification, processing a file. You do not need to know how to code. You describe what the script should do, the AI writes it, you test it together once, and it works the same way forever after.



The rule of thumb for what goes where: if a step should produce the same output every time given the same input, it belongs in a script. If a step requires judgment, taste, or reading context, it stays with the AI.

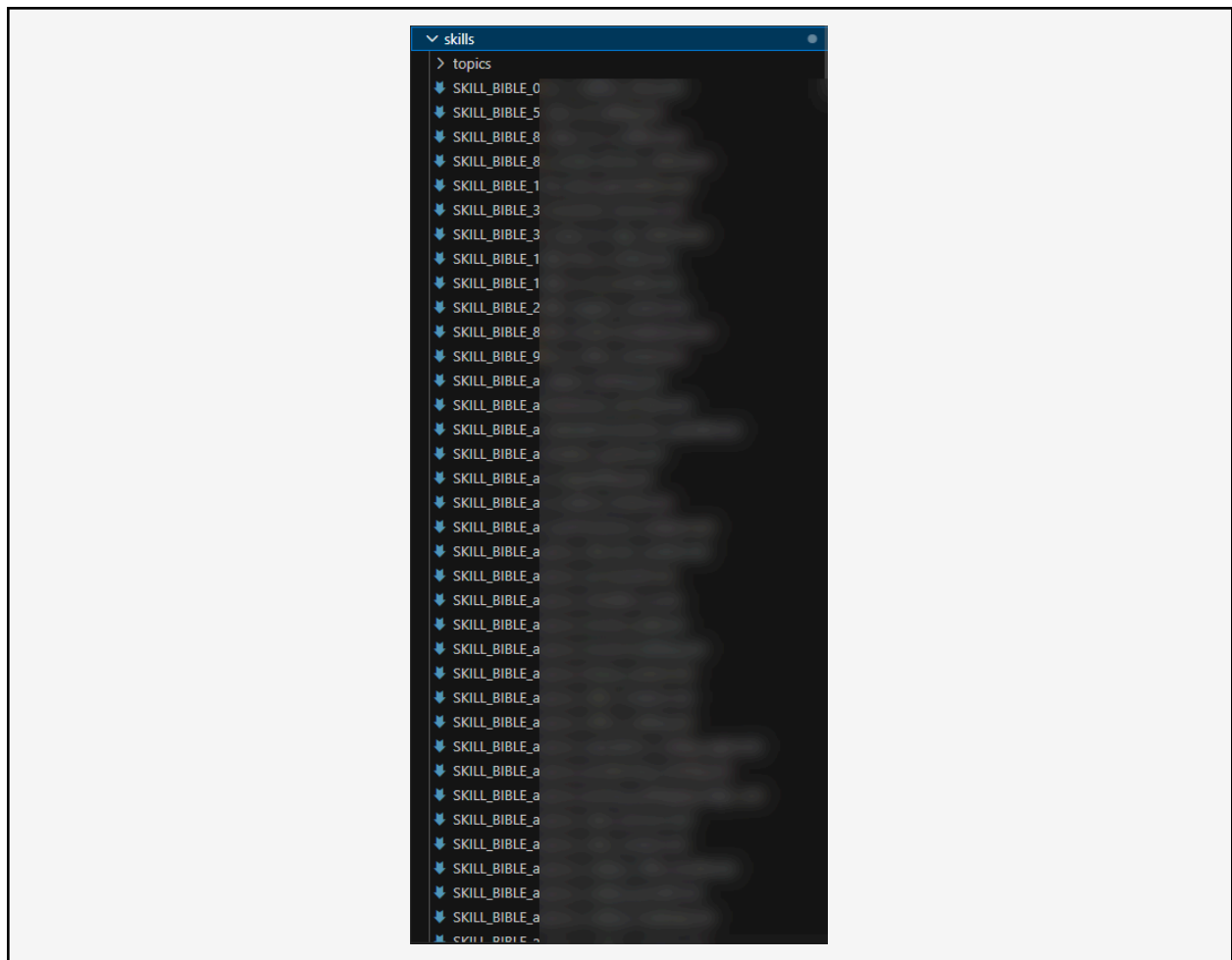
Start with the three workflows you repeat most. For most business owners that is some version of: content production, lead or client research, and a client deliverable. Do not try to build 20 SOPs in week one. Three SOPs that run clean beat 20 that half-work.

Action step: pick your three most repeated workflows. For each, do the task once WITH the AI, narrating your standards as you go. Then have the AI write the directive from that session. You just turned your habits into infrastructure.

Phase 5: Deep expertise (the skills library)

Context files cover who you are. Skill files cover what you know.

A skill file is a dense, structured extraction of domain expertise on one topic: how to write hooks, how to run discovery calls, how to structure an offer, how to price. Mine are named SKILL_BIBLE_<topic>.md and there are 297 of them.



Here is the method that built most of that library. Find the best material in your niche: a masterclass video, a course you bought, a podcast from a proven operator, your own best-performing work. Get the transcript. Have the AI extract it into a structured skill file: core principles, the exact frameworks, specific examples with numbers, common mistakes, and a quality checklist. One hour of video becomes a permanent, loadable skill.

Two rules that keep the library honest:

- **Always cite the source and date** at the top of the file. When advice conflicts later, you need to know which source and which era it came from.
- **Extract specifics, not summaries.** "Make good hooks" is worthless. "Sentence two must confirm the hook within 5 seconds, and a specific number in the first line can 2x views" is an asset. If the skill file does not contain numbers, templates, or exact phrasings, it is too shallow to change output quality.

The AI loads the relevant skill files before doing related work. Writing a sales email? It loads the email skill files. Prepping a call? It loads the sales call frameworks. Your system gets the benefit of every course you ever bought, on every single task, forever.

Action step: pick the three skills that most drive revenue in your business. Find the best source material for each. Have the AI extract each into a skill file using the structure above.

Phase 6: Client intelligence (the clients layer)

One folder per client, four files each:

```
clients/acme_corp/
├─ profile.md      - Who they are, their business, goals, stack
├─ rules.md        - Hard rules for this client (compliance, approvals,
bans)
├─ preferences.md  - Style, tone, formatting, pet peeves
└─ history.md      - Every project, outcome, and lesson, dated
```

Before any client work, the AI loads that client's folder. The result: it never uses a banned word for that client, never repeats a mistake recorded in the history file, and never needs you to re-explain the relationship.

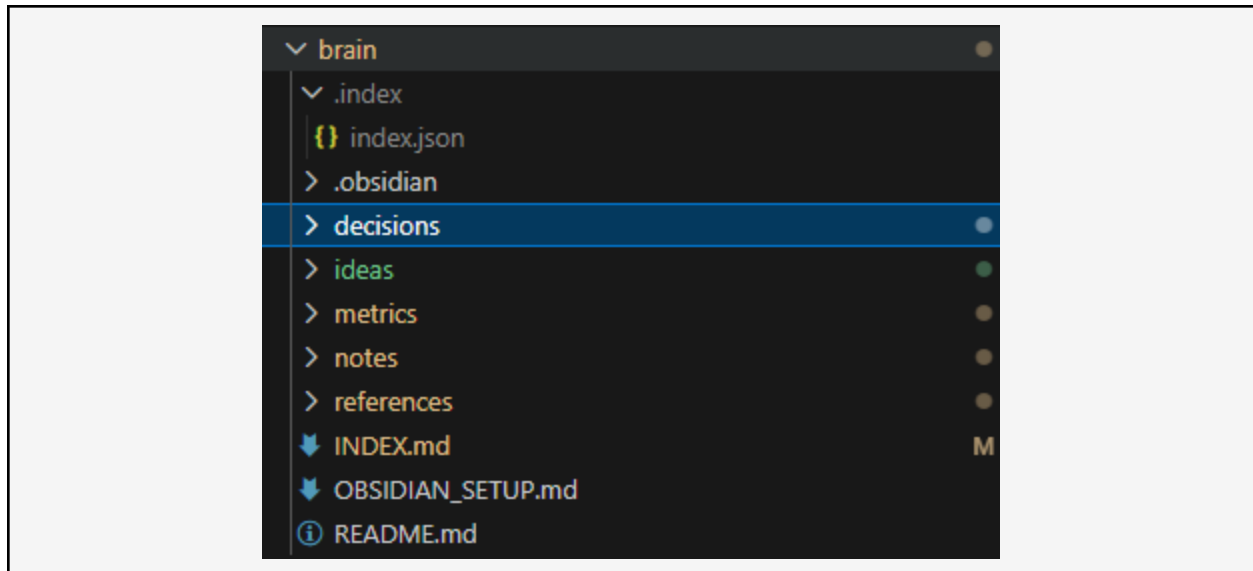
The history file is the one people skip and the one that matters most. After every meaningful client interaction, one dated entry: what happened, what was decided, what to remember. Six months later, that file is the institutional memory that normally lives in one employee's head and walks out the door when they leave.

Action step: create folders for your top five clients. Fill profiles from what you know; have the AI draft them from your call transcripts or email threads if you have them.

Phase 7: The brain itself (linked notes)

This is the layer that makes it a second brain instead of a file cabinet. The brain/ folder holds dated, linked notes in five categories:

```
brain/
├─ INDEX.md        - One line per note, the master map
├─ decisions/      - Every meaningful business decision, with reasoning
├─ notes/          - Narrative memory: initiatives, builds, events,
lessons
├─ references/     - Durable facts: playbooks, rosters, case studies,
pipelines
├─ metrics/        - Dated snapshots of the numbers
└─ ideas/          - Sparks worth keeping that are not commitments yet
```



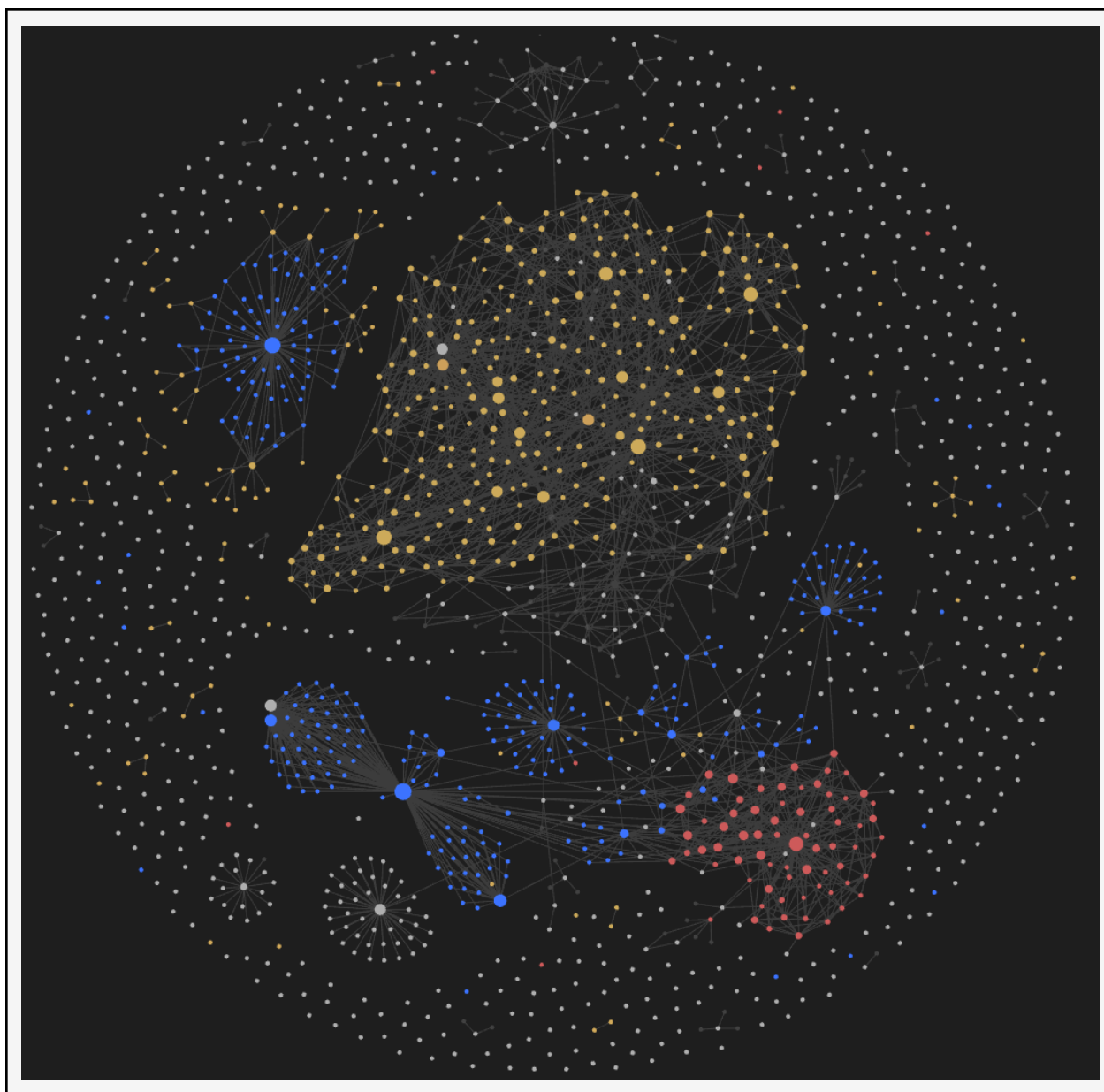
The conventions that make it work:

1. **One fact per note, dated in the filename:** 2026-06-13_productize-crm-wedge.md. Dates matter because businesses change their minds, and you need to know which note is newer.
2. **Notes link to each other** with wiki-style links, so reading one note surfaces the related ones.
3. **INDEX.md holds one line per note.** The AI reads the index first, then opens only what is relevant. This is how the brain scales past the point where anyone could read all of it.
4. **Decisions get their own category with the reasoning attached.** "We decided X because Y" is the highest-value sentence in the whole system. It stops you from re-litigating settled questions and from repeating reversed ones.

Those wiki-style links are not decoration. They are what makes this layer a brain instead of a file cabinet, and there is a tool that makes them visible: **Obsidian**.

Obsidian is a free app that treats any folder of markdown files as a vault. Point it at your business-brain folder and three things light up:

- Every wiki-style link becomes clickable, so you read the brain by following threads instead of opening files one by one
- Every note gets a backlinks panel showing every other note that references it, automatically
- The graph view draws your whole business as a network: which clients touch which decisions, which lessons cluster together, and which notes float alone with no connections



The division of labor is clean. The AI writes the notes and maintains the links; Obsidian is where you read them. I open it to follow a thread before a client call, and to spot the structural problems a file list hides: an orphan note with no connections is usually a note that needs expanding or merging. One housekeeping rule: Obsidian creates a .obsidian settings folder inside the vault. Add it to .gitignore; it is UI state, not business memory.

The links do not appear on their own. When a new frontier model dropped, one of the first jobs I gave it was wiring the entire brain together. My first version of that prompt was one line: "create as many backlinks to all relevant files as possible." It worked, but it aimed at the wrong target. Here is the refined version; copy it as is:

I read this vault in Obsidian. Go through every note in brain/ and add [[wikilinks]] connecting notes that genuinely share a client, a decision, a project, a person, or a lesson.

Rules:

- Link text must match the target note's filename exactly, so the link resolves in Obsidian.
- Only add a link where following it would teach the reader something real. Relationships, not keyword matches.
- Most notes should end up with 2 to 5 links. If a note honestly connects to nothing, leave it alone and report it instead of forcing a link.
- Add links only. Do not rewrite, trim, or improve any other content.

When you are done, report: how many links you added, which notes are orphans with no connections, and the three most surprising connections you found.

Why the rules matter. "As many as possible" produces a brain where everything links to everything, which is as useless as one where nothing does. The rewrite caps the ambition (real relationships only), protects the content (add links only, so the git diff is pure and reviewable), and turns the leftovers into a to-do list (the orphan report). Run it once your brain reaches about 20 notes, then rerun it after every bulk import from sources/.

Word Count: ~1,400

Version: 1.0

<!-- skills-graph:start -->

Topics

- [Cold Email & Outbound](#)
- [Email Marketing & Lifecycle Flows](#)

Related Skill Bibles

- [Email Deliverability Ecom](#)
- [Cold Email Infrastructure](#)
- [Email Deliverability Mastery](#)
- [Klaviyo Setup](#)

<!-- skills-graph:end -->

Now the shortcut that saves you months: **do not write your history by hand. Feed the AI your raw exports.** Slack or Teams history, call transcripts, email threads, old proposals. Drop them in a sources/ folder and have the AI mine them into dated brain notes. My brain's company timeline, client roster history, and founding journal were all extracted from Slack exports and call transcripts, not written from memory.

One maintenance habit: every few weeks, have the AI run a contradiction audit. It reads the brain looking for notes that disagree with each other (old pricing versus new pricing, stale team rosters, reversed decisions), flags them, and you rule on each one. The brain stays trustworthy because it gets audited like a real system.

Action step: create the brain folders and INDEX.md. Write your first three notes today: one decision you made recently with the reasoning, one reference fact you keep re-explaining, one lesson from the last month. Open the folder as a vault in Obsidian and watch the graph fill in as notes land. Then gather your raw exports into sources/ and schedule a session where the AI mines them.

Phase 8: Make it self-improving

Two mechanisms turn the folder from a static wiki into a system that compounds.

Self-annealing. A standing rule in CLAUDE.md: after every task, if an error occurred, fix the script and update the directive; if a better approach was found, update the skill file; if a new edge case appeared, add it to the SOP. Nothing breaks the same way twice, because every failure becomes an edit to the system. This is the "1 percent better every day" value made mechanical.

Persistent memory. Give the AI a place to write down what it learns about working with you: your preferences, corrections you have given, ongoing projects, facts that are not in any file yet. Claude Code has this built in. The effect is that corrections stick. You tell it once that reaction videos should be 15 to 25 seconds, and every future session already knows.

Quality gates. Every directive ends with a checklist the output must pass before it reaches you. Content gets checked against the brand voice file. Client work gets checked against the client rules file. Anything with numbers gets checked against the rule that numbers are never invented. You review the work, but the system catches the routine failures before you see them.

Action step: add the self-annealing protocol to CLAUDE.md as a standing rule. Add a quality gate section to each of your three directives.

Phase 9: Delivery and integrations (optional, do this last)

Once the core system works, connect it to where work actually gets delivered:

- **Document delivery:** a script that turns markdown outputs into Google Docs for clients and teammates
- **Notifications:** a script that posts to Slack when work is done
- **Scheduled runs:** recurring tasks (a daily news brief, a weekly content plan) triggered on a schedule
- **A dashboard:** if you have a team, a simple password-protected web app that lists every workflow and its documentation

None of this is required to get value. I ran the system for weeks with nothing but files and the AI. Add integrations when a real bottleneck demands them, not before.

Part 3: The seven-day build order

Do not build alphabetically. Build in the order that compounds:

- **Day 1:** Folder, git, CLAUDE.md first draft, the four context files (use the record-and-transcribe method)
- **Day 2:** First directive plus first script, for your single most repeated workflow. Run it end to end once.
- **Day 3:** Two more directives. Start the clients folder with your top five.
- **Day 4:** First three skill files from your best source material.
- **Day 5:** Brain structure plus first ten notes. Start with decisions; they are the highest value per minute of effort.
- **Day 6:** Gather raw exports into sources/ and run the first mining session. This is the day the brain gets deep.
- **Day 7:** Self-annealing rules, quality gates, first contradiction check. Then use the system on real work and let the corrections improve it.

After day 7 you stop building the system and start using it. It grows as a side effect of the work.

Part 4: The mistakes that kill second brains

Building the perfect structure before putting anything in it. An empty taxonomy is worth nothing. Ten messy but real notes beat a beautiful empty folder tree.

Writing summaries instead of specifics. If a note or skill file has no numbers, no names, no dates, and no exact phrasings, the AI cannot do anything with it that it could not already do without it.

Skipping dates. An undated fact becomes a landmine the first time the business changes its mind. Date everything, in the filename.

Letting the AI invent facts. Put it in writing as a standing rule: no fabricated numbers, results, or client names, ever. Placeholders plus a question beat confident fiction every time.

Treating it as an archive instead of an operator. The test of the system is whether the AI can pick up a task cold, load the right files, and produce work that passes your quality bar. Store less, operationalize more.

Never auditing. A brain that is never checked for contradictions slowly becomes a brain you cannot trust, and then you stop using it. Audit on a schedule.

What you will have at the end

A folder that contains your positioning, your voice, your values, your SOPs, your client relationships, your decision history, and your accumulated expertise, all in plain text, all readable by any AI model that exists now or ships later.

Models will keep getting replaced. The one that helped build my system is on its way out as I write this, and the system does not care. That is the whole point. The model is the employee. **The folder is the company.**

Build the folder.

Next step

I run DobotAI Academy, a community where business owners become AI Technical Operators and build systems exactly like this one, with the templates, the walkthroughs, and direct feedback on your build.

If you got this guide from my content, the link is in my bio. Bring your first three directives and I will tell you what to fix.

Or click here: <https://www.skool.com/dobotaiacademy/about>

Doby Lanete

DobotAI - The Last AI Partner You'll Ever Need